CS202, Spring 2026

Homework 3 - Hashing

Due: 23:55, 20/04/2026

Before you start your homework, please read the following instructions carefully:

FAILURE TO FULFIL ANY OF THE FOLLOWING REQUIREMENTS WILL RESULT IN A GRADE SCORE OF 0 (zero) WITHOUT ANY CHANCE OF REDEMPTION.

- See the course page for any late submission policies and Honor Code for Assignments.
- Upload your solutions in a single ZIP archive using the Moodle submission form. Name the file as studentID_name_surname_hw3.zip.
- Your ZIP archive should contain **only** the following files:
    - **studentID_name_surname_hw3.pdf**, the file containing the answers to Questions 1, 2.
    - In this assignment, you must have separate interface and implementation files (i.e., separate .h
    - and .cpp files) for your class. Your class name MUST BE **HashTable** and your file names MUST BE
    - **HashTable.h** and **HashTable.cpp**. You have to implement **Question3.cpp**, **Question4.cpp** and **Question5.cpp** to solve and test question 3-5.Note that you may write additional class(es) in your solution.
    - Your .cpp, .h , .pdf files, and **Makefile** , which produces executables for each subtask (No Makefile results in 50% deduction in points).
    - Do not forget to put your name, student id, and section number in all of these files. Comment your implementation well. Add a header (see below) to the beginning of each file:

```
/**
 * Title: Hashing
 * Author : Name & Surname
 * ID: 12345678
 * Section : 1
 * Homework : 3
 * Description : description of your code
 */
```

    - Do not put any unnecessary files, such as the auxiliary files generated from your preferred IDE.

    - Please do not use **Turkish** letters in your file and folder names.
- Your code must compile.
- Your code must be complete.
- You ARE NOT ALLOWED to use any data structure or algorithm related function from the C++ standard template library (STL) or any other external libraries.
- We will test your code using the **google test library**. Therefore, the output message for each operation in Question 3, 4 and 5 MUST match the format shown in the output of the example code.
- Your code must run on the **dijkstra.cs.bilkent.edu.tr** server.
- For any question related to the homework, contact your TA: *pinar.gul@bilkent.edu.tr* or you can ask your questions on the forum.

Question 1 - 8 points

Assume that we have a hash table of size 11 (index range is 0..10), and we use the hash function

$$h(x) = x \bmod 11$$

Insert the following keys in the given order:

`22, 1, 13, 44, 24, 29, 33, 14`

Draw the final hash tables for each of the following collision handling methods:

- open addressing with linear probing
- open addressing with quadratic probing

You must clearly show the final table contents.
You MUST have a handwritten answer for this question.

Question 2 - 12 points

a) Suppose we use a hash table with separate chaining. The initial table size is 5, and the hash function is

$$h(x) = x \bmod m$$

where **m** is the current table size.

We rehash whenever the load factor becomes strictly greater than **`1.0`,** and after rehashing, the new table size becomes the next prime number greater than **2m**.

Insert the following keys in the given order:

`7, 12, 3, 18, 25, 14, 9, 21, 30, 4, 16, 28`

For each insertion:

1. compute the bucket index,
2. show the table contents,
3. indicate whether rehashing occurs.

Draw the final hash table after all insertions.

b) Assume that m is **17** and this time we use **linear probing**. Initial table is empty, after inserting the keys `18, 39, 35, 52, x` in this order, the final positions are:

- `18` is stored at index `1`
- `39` is stored at index `5`
- `35` is stored at index `2`
- `52` is stored at index `3`
- `x` is stored at index `4`

Find the **smallest possible positive integer value** of `x`.

You **MUST** have a handwritten answer for this question.

Question 3 - 22 points

You are given a string **S** consisting of lower-case English letters. You will encode this string using the following dictionary-based compression rule.

Initially, the dictionary contains all single lower-case letters:

- **`"a"`** has code **`0`**
- **`"b"`** has code **`1`**
- ...
- **`"z"`** has code **`25`**

The next available code is **`26`**.

Then, starting from the beginning of the string, repeatedly do the following:

1. Find the **longest prefix** of the remaining unencoded part of the string that already exists in the dictionary.

2. Output the code of this prefix.
3. If there is still a next character after this prefix, let the prefix be ***p*** and the next character be ***c***. Insert the string ***p+c*** into the dictionary with the next available code, **if it is not already in the dictionary**.
4. Continue from the first unencoded character after ***p***.

Your task is to output the sequence of codes produced by this process.

Input format：The first line contains a single string **S**, consisting only of lower-case English letters.

Output format : Print the generated codes in one line, separated by one space.

Sample Input (input3.txt):

aaabbbbbbaabaaba

Sample Output (output3.txt):

0 26 1 28 29 27 31

Initially, the dictionary contains all one-letter strings from `"a"` to `"z"`.

For the input `aaabbbbbbaabaaba`:

- The longest prefix at the beginning is `"a"` → output `0`, insert `"aa"` with code `26`
- The next longest prefix is `"aa"` → output `26`, insert `"aab"` with code `27`
- The next longest prefix is `"b"` → output `1`, insert `"bb"` with code `28`
- The next longest prefix is `"bb"` → output `28`, insert `"bbb"` with code `29`
- The next longest prefix is `"bbb"` → output `29`, insert `"bbba"` with code `30`
- The next longest prefix is `"aab"` → output `27`, insert `"aaba"` with code `31`
- The remaining prefix is `"aaba"` → output `31`

Therefore, the encoded output is:

```
0 26 1 28 29 27 31
```

Important

-This question **MUST** be solved using **hashing / hash tables**.

-You must use a hash table to store dictionary entries of the form
(string -> code).

-If you try to solve this question using data structures other than hash tables for the main dictionary operations, you will get 0 from this part.

- You MUST implement your hash table using separate chaining.

The name of the input file will be provided as a command-line argument to your program. Thus, your program should run using two command-line arguments. Thus, the application interface is simple and given as follows:

```
username@dijkstra:~>./Question3 <input_filename> <output_filename>
```

Assuming that you have an executable called "Question3", this command calls the executable with one command line argument, the name of the file from which your program reads the input string.

Note that outputfile name is optional.

Question 4 - 25 points

You are given **N** text documents. Each document consists of characters, and only alphabetical characters (a-z, A-Z) are considered as part of a word. All other characters are separators. Words are case-insensitive, so for example Hash, hash, and HASH are considered the same word.

Your task is to preprocess all documents and answer a query. The query consists of one line containing one or more words. You must output the names of the documents that contain **all** queried words, together with the number of occurrences of each queried word in that document.

For example, if the query is: **data structure**,

then a document is reported only if it contains both **data** and **structure**. For each such document, you print how many times data appears and how many times structure appears in that document.

You must preprocess the documents using a **hash table**. For each distinct word, your hash table must store the list of documents in which the word appears, together with its frequency in each document.

Input format:

- The first line contains an integer **N**, the number of documents.
- The next **N** lines contain the file names of the documents.
- The last line contains the query words as a single line.

Word processing rules:

- Only alphabetical characters are part of words.
- Any non-alphabetical character is treated as a separator.
- Upper-case and lower-case letters are treated as the same.
- If the same word appears multiple times in a document, its frequency must be counted.
- Query words must also be processed using the same rules.

Output format:

If there exists at least one document containing all query words, print one line for each such document in the following format:

`in Document <document_name>, <word1> found <count1> times, <word2> found <count2> times, ... .`

The queried words must be printed in the same order as they appear in the processed query.

If no document contains all query words, print:

`No document contains the given query.`

Sample Input1 (input4.txt):
2
a.txt
b.txt
above close

Sample Output1 (output4.txt):
in Document a.txt, above found 12 times, close found 1 times.

Sample Input1 (input4.txt):
3
d.txt
e.txt
f.txt
a the-to as

Sample Output1 (output4.txt):
in Document d.txt, a found 4 times, the found 12 times, to found 4 times, as found 1 times.
in Document e.txt, a found 4 times, the found 9 times, to found 3 times, as found 1 times.
in Document f.txt, a found 7 times, the found 33 times, to found 20 times, as found 3 times.

In Sample Input 2, the query line is processed by treating non-alphabetical characters as separators. Therefore, the line

```
a the-to as
```

is interpreted as the query words:

- `a`
- `the`
- `to`
- `as`

Only the documents containing all four words are reported.

Important

-This question **MUST** be solved using **hash tables**.

-If you try to solve this question using data structures other than hash tables for the main preprocessing and query mechanism, you will get 0 from this part.

-Your solution must preprocess the documents efficiently and answer the query using the preprocessed hash-based index.

-The document files listed in the input will be provided as part of the homework support files.

-The document files referenced in the input are expected to be in the same working directory as the executable. For convenience, while testing, you may place the input file and all referenced document files in the same folder as your executable.

-The name of the input file will be provided as a command-line argument to your program.

```
username@dijkstra:~>./Question4 <input_filename> <output_filename>
```

Assuming that you have an executable called "Question4", this command calls the executable with one command line argument, the name of the file from which your program reads the number of documents, document file names, and the query line.

Note that outputfile name is optional.

Question 5 - 33 points

A jewelry store keeps records of bracelets made of colored beads. Since the bracelets are circular, the same bracelet may be recorded starting from any bead. Also, a bracelet may be turned over, in which

case the reading direction is reversed. For convenience, each bead color is represented by a symbol. These symbols are **color codes**, not printed letters on the bracelet.

You are given ***n*** unique bracelet records. Each record is a string over the alphabet

$$\{R,G,B,Y,W,K\}$$

where each symbol denotes one bead color.

Two bracelet records are considered to represent the same bracelet type if one can be obtained from the other by:

1) Any number of cyclic shifts, and

2) Optionally one reversal before the shifts.

For example, the records **RGBYW**, **GBYWR**, and **WYBGR** all represent the same bracelet type. The shop wants to organize the bracelets by type. Since different bracelet types must be stored separately, the shop will buy one box for each distinct bracelet type.

Part I

Determine the number of distinct bracelet types. This is also the number of boxes the shop must buy. Therefore, find the number of boxes that the shop will buy.

Part II

After grouping the bracelets by type, the shop wants all records in the same group to be written in a common orientation. Cyclic shifts are free, but reversing a bracelet record is considered costly.

For each bracelet type, choose a common orientation so that all records of that type can be converted to it using cyclic shifts and, if necessary, reversal first. Determine the minimum total number of reversals required over all bracelet types.

Input format:

- The first line contains an integer **n**, the number of bracelet records.
- The next **n** lines each contain one string over the alphabet {R,G,B,Y,W,K}

Output format:

Print two lines:

- In the first line, print the number of distinct bracelet types.
- In the second line, print the minimum total number of reversals required.

Sample Input1:
2
RGBYW
WYBGR

Sample Output1:
1
1

Sample Input1:
4
BRRR
RRRB
BGY
YGB

Sample Output1:
2
1

Explanation for sample 1:

The two records represent the same bracelet type. Therefore, the shop needs only 1 box.

However, the two records do not have the same reading direction. One reversal is enough to make them share a common orientation, so the minimum total number of reversals is 1.

Explanation for sample 2:

The records BRRR and RRRB represent the same bracelet type because one is a cyclic shift of the other. No reversal is needed for them. The records BGY and YGB also represent the same bracelet type, but one reversal is needed to make them share a common orientation. So there are 2 distinct bracelet types in total, and the minimum total number of reversals is 1.

Important

-This question **MUST** be solved using **hashing**.

-If you try to solve this question using data structures other than hash tables or hashing for the main grouping mechanism, you will get 0 from this part.

-The name of the input file will be provided as a command line argument to your program.

```
username@dijkstra:~>./Question5 <input_filename> <output_filename>
```

Assuming that you have an executable called "Question5", this command calls the executable with one command line argument, the name of the file from which your program reads the number of bracelet records and the records themselves.

Note that outputfile name is optional.